Assignment 2: Building A Task Execution Library from the Ground Up 从头开始构建任务执行库

姓名：温舒盈　　学号：71066002

（翻译部分基本依靠谷歌网页翻译插件实现）

Overview 概述

Everyone likes to complete tasks quickly, and in this assignment we are asking you to do just that! You will implement a C++ library that executes tasks provided by an application as efficiently as possible on a multi-core CPU.
每个人都喜欢快速完成任务，在这个任务中，我们要求你做到这一点！您将实现一个C++库，该库在多核CPU上尽可能高效地执行应用程序提供的任务。

In the first part of the assignment, you will implement a version of the task execution library that supports bulk (data-parallel) launch of many instances of the same task. This functionality is similar to the ISPC task launch behavior you used to parallelize code across cores in Assignment 1.
在作业的第一部分中，您将实现一个版本的任务执行库，该库支持同一任务的多个实例的批量（数据并行）启动。此功能类似于您在Assignment 1中用于跨核并行化代码的ISPC任务启动行为。

In the second part of the assignment, you will extend your task runtime system to execute more complex *task graphs*, where the execution of tasks may depend on the results produced by other tasks. These dependencies constrain which tasks can be safely run in parallel by your task scheduling system. Scheduling execution of data-parallel task graphs on a parallel machine is a feature of many popular parallel runtime systems ranging from the popular Thread Building Blocks library, to Apache Spark, to modern deep learning frameworks such as PyTorch and TensorFlow.
在作业的第二部分，您将扩展任务运行时系统以执行更复杂的任务图，其中任务的执行可能取决于其他任务产生的结果。这些依赖关系约束了哪些任务可以由任务调度系统安全地并行运行。在并行机上调度数据并行任务图的执行是许多流行的并行运行时系统的一个功能，从流行的Thread Building Blocks库到Apache Spark，再到现代深度学习框架，如PyTorch和TensorFlow。

This assignment will require you to:
这项任务将要求您：

- Manage task execution using a thread pool
  使用线程池管理任务执行
- Orchestrate worker thread execution using synchronization primitives such as mutexes and condition variables
  使用同步原语（如互斥锁和条件变量）来描述工作线程的执行
- Implement a task scheduler that reflects dependencies defined by a task graph
  实现反映任务图定义的依赖关系的任务调度程序
- Understand workload characteristics to make efficient task scheduling decisions
  了解工作负载特征以制定高效的任务调度决策

Wait, I Think I've Done This Before? 等等，我想我以前做过这个?

You may have already created thread pools and task execution libraries in classes such as CS107 or CS111. However, the current assignment is a unique opportunity to better understand these systems. You will implement multiple task execution libraries, some without thread pools and some with different types of thread pools. By implementing multiple task scheduling strategies and comparing their performance on difference workloads, you will better understand the implications of key design choices when creating a parallel system.
您可能已经在CS107或CS 111等类中创建了线程池和任务执行库。然而，目前的任务是一个独特的机会，以更好地了解这些系统。您将实现多个任务执行库，有些没有线程池，有些具有不同类型的线程池。通过实现多任务调度策略并比较它们在不同工作负载上的性能，您将更好地理解创建并行系统时关键设计选择的含义。

Environment Setup 环境设置

We will be grading this assignment on an Amazon AWS `m6i.2xlarge` instance - we provide instructions for setting up your VM here. Please ensure your code works on this VM as we will be using this for performance testing and grading.
我们将在Amazon AWS `m6i.2xlarge` 实例上对此作业进行评分-我们在此处提供设置VM的说明。请确保您的代码在此虚拟机上工作，因为我们将使用此虚拟机进行性能测试和评级。

The assignment starter code is available on Github. Please clone the Assignment 2 starter code using:
代码可以在Github上找到。请使用以下命令克隆Assignment 2启动代码：

```
git clone https://github.com/stanford-cs149/asst2.git
```

IMPORTANT: DO NOT modify the provided `Makefile`. Doing so may break our grading script.
重要提示：请勿修改提供的 `Makefile` 。这样做可能会破坏我们的评分脚本。

Part A: Synchronous Bulk Task Launch 同步批量任务启动

In Assignment 1, you used ISPC's task launch primitive to launch N instances of an ISPC task (`launch[N] myISPCFunction()`). In the first part of this assignment, you will implement similar functionality in your task execution library.
在作业1中，您使用ISPC的任务启动原语启动了ISPC任务的N个实例（`launch[N] myISPCFunction()`）。在本作业的第一部分中，您将在任务执行库中实现类似的功能。

To get started, get acquainted with the definition of `ITaskSystem` in `itasksys.h`. This abstract class defines the interface to your task execution system. The interface features a method `run()`, which has the following signature:
要开始，请熟悉 `itasksys.h` 中的 `ITaskSystem` 的定义。这个抽象类定义了任务执行系统的接口。该接口具有方法 `run()` ，其具有以下签名：

```
virtual void run(IRunnable* runnable, int num_total_tasks) = 0;
```

`run()` executes `num_total_tasks` instances of the specified task. Since this single function call results in the execution of many tasks, we refer to each call to `run()` as a *bulk task launch*.
`run()` 执行指定任务的 `num_total_tasks` 个实例。由于这个单一的函数调用会导致许多任务的执行，我们将每个对 `run()` 的调用称为批量任务启动。

The starter code in `tasksys.cpp` contains a correct, but serial, implementation of `TaskSystemSerial::run()` which serves as an example of how the task system uses the `IRunnable` interface to execute a bulk task launch. (The definition of `IRunnable` is in `itasksys.h`) Notice how in each call to `IRunnable::runTask()` the task system provides the task a current task identifier (an integer between 0 and `num_total_tasks`), as well as the total number of tasks in the bulk task launch. The task's implementation will use these parameters to determine what work the task should do.
`tasksys.cpp` 中的启动器代码包含 `TaskSystemSerial::run()` 的正确但串行的实现，其用作任务系统如何使用 `IRunnable` 接口来执行批量任务启动的示例。(The `IRunnable` 的定义在 `itasksys.h` 中）注意在对 `IRunnable::runTask()` 的每次调用中，任务系统如何向任务提供当前任务标识符（0和 `num_total_tasks` 之间的整数），以及批量任务启动中的任务总数。任务的实现将使用这些参数来确定任务应该做什么工作。

One important detail of `run()` is that it must execute tasks synchronously with respect to the calling thread. In other words, when the call to `run()` returns, the application is guaranteed that the task system has completed execution of **all tasks** in the bulk task launch. The serial implementation of `run()` provided in the starter code executes all tasks on the calling thread and thus meets this requirement.
`run()` 的一个重要细节是它必须相对于调用线程同步执行任务。换句话说，当对 `run()` 的调用返回时，应用被保证任务系统已经完成批量任务启动中的所有任务的执行。启动器代码中提供的 `run()` 的串行实现执行调用线程上的所有任务，因此满足此要求。

Running Tests 运行测试

The starter code contains a suite of test applications that use your task system. For a description of the test harness tests, see `tests/README.md`, and for the test definitions themselves, see `tests/tests.h`. To run a test, use the `runtasks` script. For example, to run the test called `mandelbrot_chunked`, which computes an image of a Mandelbrot fractal using a bulk launch of tasks that each process a continuous chunk of the image, type:
启动代码包含一套使用任务系统的测试应用程序。有关测试线束测试的描述，请参见 `tests/README.md` ，有关测试定义本身，请参见 `tests/tests.h` 。要运行测试，请使用 `runtasks` 脚本。例如，要运行名为 `mandelbrot_chunked` 的测试，该测试使用批量启动的任务（每个任务处理图像的连续块）计算Mandelbrot分形的图像，请键入：

```
./runtasks -n 8 mandelbrot_chunked
```

The different tests have different performance characteristics -- some do little work per task, others perform significant amounts of processing. Some tests create large numbers of tasks per launch, others very few. Sometimes the tasks in a launch all have similar compute cost. In others, the cost of tasks in a single bulk launch is variable. We have described most of the tests in `tests/README.md`, but we encourage you to inspect the code in `tests/tests.h` to understand the behavior of all tests in more detail.
不同的测试具有不同的性能特征--有些测试在每个任务中只做很少的工作，而另一些则执行大量的处理。有些测试在每次启动时会创建大量的任务，而有些则很少。有时，启动中的所有任务都具有类似的

计算成本。在其他情况下，单次批量发射的任务成本是可变的。我们已经在 `tests/README.md` 中描述了大多数测试，但我们鼓励您检查 `tests/tests.h` 中的代码，以更详细地了解所有测试的行为。

One test that may be helpful to debug correctness while implementing your solution is `simple_test_sync`, which is a very small test that should not be used to measure performance but is small enough to be debuggable with print statements or debugger. See function `simpleTest` in `tests/tests.h`.
在实现解决方案时，可能有助于调试正确性的一个测试是 `simple_test_sync` ，这是一个非常小的测试，不应用于测量性能，但足够小，可以使用print语句或调试器进行调试。参见 `tests/tests.h` 中的功能 `simpleTest` 。

We encourage you to create your own tests. Take a look at the existing tests in `tests/tests.h` for inspiration. We have also included a skeleton test composed of `class YourTask` and function `yourTest()` for you to build on if you so choose. For the tests you do create, make sure to add them to the list of tests and test names in `tests/main.cpp`, and adjust the variable `n_tests` accordingly. Please note that while you will be able to run your own tests with your solution, you will not be able to compile the reference solution to run your tests.
我们鼓励您创建自己的测试。看看 `tests/tests.h` 中的现有测试以获得灵感。我们还包含了一个由 `class YourTask` 和函数 `yourTest()` 组成的框架测试，如果您选择的话，可以在此基础上进行构建。对于您创建的测试，请确保将它们添加到 `tests/main.cpp` 中的测试和测试名称列表中，并相应地调整变量 `n_tests` 。请注意，虽然您可以使用解决方案运行自己的测试，但您将无法编译参考解决方案来运行测试。

The `-n` command-line option specifies the maximum number of threads the task system implementation can use. In the example above, we chose `-n 8` because the CPU in the AWS instance features eight execution contents. The full list of tests available to run is available via command line help (`-h` command line option).
`-n` 命令行选项指定任务系统实现可以使用的最大线程数。在上面的示例中，我们选择 `-n 8` 是因为AWS实例中的CPU具有八个执行内容。可通过命令行帮助（ `-h` 命令行选项）获得可运行的测试的完整列表。

The `-i` command-line options specifies the number of times to run the tests during performance measurement. To get an accurate measure of performance, `./runtasks` runs the test multiple times and records the *minimum* runtime of several runs; In general, the default value is sufficient---Larger values might yield more accurate measurements, at the cost of greater test runtime.
`-i` 命令行选项指定在性能测量期间运行测试的次数。为了获得准确的性能度量， `./runtasks` 多次运行测试并记录几次运行的最小运行时间;通常，默认值就足够了-较大的值可能会产生更准确的度量，但测试运行时间会更长。

In addition, we also provide you the test harness that we will use for grading performance:
此外，我们还为您提供用于性能分级的测试线束：

```
>>> python ../tests/run_test_harness.py
```

The harness has the following command line arguments,
该工具具有以下命令行参数，

```
>>> python run_test_harness.py -h
usage: run_test_harness.py [-h] [-n NUM_THREADS]
                           [-t TEST_NAMES [TEST_NAMES ...]] [-a]

Run task system performance tests
```

```
optional arguments:
  -h, --help            show this help message and exit
  -n NUM_THREADS, --num_threads NUM_THREADS
                        Max number of threads that the task system can use. (8
                        by default)
  -t TEST_NAMES [TEST_NAMES ...], --test_names TEST_NAMES [TEST_NAMES ...]
                        List of tests to run
  -a, --run_async       Run async tests
```

It produces a detailed performance report that looks like this:
它生成一个详细的性能报告，如下所示：

```
>>> python ../tests/run_test_harness.py -t super_light super_super_light
python ../tests/run_test_harness.py -t super_light super_super_light
================================================================================
Running task system grading harness... (2 total tests)
  - Detected CPU with 8 execution contexts
  - Task system configured to use at most 8 threads
================================================================================
================================================================================
Executing test: super_super_light...
Reference binary: ./runtasks_ref_linux
Results for: super_super_light
                                      STUDENT   REFERENCE   PERF?
[Serial]                              9.053     9.022       1.00  (OK)
[Parallel + Always Spawn]             8.982     33.953      0.26  (OK)
[Parallel + Thread Pool + Spin]       8.942     12.095      0.74  (OK)
[Parallel + Thread Pool + Sleep]      8.97      8.849       1.01  (OK)
================================================================================
Executing test: super_light...
Reference binary: ./runtasks_ref_linux
Results for: super_light
                                      STUDENT   REFERENCE   PERF?
[Serial]                              68.525    68.03       1.01  (OK)
[Parallel + Always Spawn]             68.178    40.677      1.68  (NOT OK)
[Parallel + Thread Pool + Spin]       67.676    25.244      2.68  (NOT OK)
[Parallel + Thread Pool + Sleep]      68.464    20.588      3.33  (NOT OK)
================================================================================
Overall performance results
[Serial]                              : All passed Perf
[Parallel + Always Spawn]             : Perf did not pass all tests
[Parallel + Thread Pool + Spin]       : Perf did not pass all tests
[Parallel + Thread Pool + Sleep]      : Perf did not pass all tests
```

In the above output `PERF` is the ratio of your implementation's runtime to the reference solution's runtime. So values less than one indicate that your task system implementation is faster than the reference implementation.
在上面的输出中， `PERF` 是您的实现的运行时间与参考解决方案的运行时间之比。因此，小于1的值表示任务系统实现比参考实现快。

Mac users: While we provided reference solution binaries for both part a and part b, we will be testing your code using the linux binaries. Therefore, we recommend you check your implementation in the AWS instance before submitting. If you are using a newer Mac with an M1 chip, use the `runtasks_ref_osx_arm` binary when testing locally. Otherwise, use the `runtasks_ref_osx_x86` binary.
Mac用户：虽然我们为a部分和B部分提供了参考解决方案二进制文件，但我们将使用Linux二进制文件测试您的代码。因此，我们建议您在提交之前检查AWS实例中的实现。如果您使用的是带有M1芯片的较新Mac，请在本地测试时使用 `runtasks_ref_osx_arm` 二进制文件。否则，使用 `runtasks_ref_osx_x86` 二进制文件。

What You Need To Do 你需要做的

Your job is to implement a task execution engine that efficiently uses your multi-core CPU. You will be graded on both the correctness of your implementation (it must run all the tasks correctly) as well as on its performance. This should be a fun coding challenge, but it is a non-trivial piece of work. To help you stay on the right track, to complete Part A of the assignment, we will have you implement multiple versions of the task system, slowly increasing in complexity and performance of your implementation. Your three implementations will be in the classes defined in `tasksys.cpp/.h`.
你的工作是实现一个任务执行引擎，有效地使用你的多核CPU。你将根据你的实现的正确性（它必须正确地运行所有的任务）以及它的性能来评分。这应该是一个有趣的编码挑战，但它是一个不平凡的工作。为了帮助您保持正确的轨道，完成任务的A部分，我们将让您实现任务系统的多个版本，慢慢增加您实现的复杂性和性能。您的三个实现将在 `tasksys.cpp/.h` 中定义的类中。

- `TaskSystemParallelSpawn`
- `TaskSystemParallelThreadPoolSpinning`
- `TaskSystemParallelThreadPoolSleeping`

Implement your part A implementation in the `part_a/` sub-directory to compare to the correct reference implementation (`part_a/runtasks_ref_\*`).
在 `part_a/` 子目录中实现您的A部分实现，以与正确的参考实现（ `part_a/runtasks_ref_\*` ）进行比较。

Pro tip: Notice how the instructions below take the approach of "try the simplest improvement first". Each step increases the complexity of the task execution system's implementation, but on each step along the way you should have a working (fully correct) task runtime system.
专业提示：注意下面的说明是如何采取"先尝试最简单的改进"的方法的。每一步都增加了任务执行系统实现的复杂性，但是在沿着的每一步中，您都应该有一个工作的（完全正确的）任务运行时系统。

We also expect you to create at least one test, which can test either correctness or performance. See the Running Tests section above for more information.
我们还希望您至少创建一个测试，可以测试正确性或性能。有关详细信息，请参阅上面的"运行测试"部分。

Step 1: Move to a Parallel Task System 第一步：转移到并行任务系统

In this step please implement the class `TaskSystemParallelSpawn`.
在这一步中，请实现类 `TaskSystemParallelSpawn` 。

The starter code provides you a working serial implementation of the task system in `TaskSystemSerial`. In this step of the assignment you will extend the starter code to execute a bulk task launch in parallel.
starter代码为您提供了 `TaskSystemSerial` 中任务系统的工作串行实现。在任务的这一步中，您将扩

展启动器代码以并行执行批量任务启动。

- You will need to create additional threads of control to perform the work of a bulk task launch. Notice that `TaskSystem`'s constructor is provided a parameter `num_threads` which is the **maximum number of worker threads** your implementation may use to run tasks.
  您将需要创建额外的控制线程来执行批量任务启动的工作。请注意， `TaskSystem` 的构造函数提供了一个参数 `num_threads` ，这是您的实现可以用于运行任务的最大工作线程数。
- In the spirit of "do the simplest thing first", we recommend that you spawn worker threads at the beginning of `run()` and join these threads from the main thread before `run()` returns. This will be a correct implementation, but it will incur significant overhead from frequent thread creation.
  本着“先做最简单的事情”的精神，我们建议您在 `run()` 开始时生成工作线程，并在 `run()` 返回之前从主线程加入这些线程。这将是一个正确的实现，但它会因频繁的线程创建而产生大量开销。

> 最终代码：
>
> `tasksys.cpp`：

```
TaskSystemParallelSpawn::TaskSystemParallelSpawn(int num_threads) :
ITaskSystem(num_threads), num_threads_(num_threads) {
threads_pool_ = std::make_unique<std::thread[]>(num_threads);
}

void TaskSystemParallelSpawn::threadRun(IRunnable* runnable, int
num_total_tasks, std::mutex* mutex, int* curr_task) {
int turn = -1;
while (turn < num_total_tasks) {
   mutex->lock();
   turn = *curr_task;
   *curr_task += 1;
   mutex->unlock();
   if (turn >= num_total_tasks) {
       break;
   }
   runnable->runTask(turn, num_total_tasks);
}
}

void TaskSystemParallelSpawn::run(IRunnable* runnable, int
num_total_tasks) {
std::mutex mutex;
int curr_task = 0;
for (int i = 0; i < num_threads_; i++) {
   threads_pool_[i] = std::thread(&TaskSystemParallelSpawn::threadRun,
this, runnable, num_total_tasks, &mutex, &curr_task);
}
for (int i = 0; i < num_threads_; i++) {
   threads_pool_[i].join();
}
}
```

> `tasksys.h`：

```
class TaskSystemParallelSpawn : public ITaskSystem {
```

```
public:
TaskSystemParallelSpawn(int num_threads);
~TaskSystemParallelSpawn();
const char* name() override;
void run(IRunnable* runnable, int num_total_tasks) override;
TaskID runAsyncWithDeps(IRunnable* runnable, int num_total_tasks,
                        const std::vector<TaskID>& deps) override;
void sync() override;

private:
void threadRun(IRunnable* runnable, int num_total_tasks, std::mutex*
mutex, int* curr_task);
std::unique_ptr<std::thread[]> threads_pool_;
int num_threads_;
};

/*
* TasksState: This class is used to record the state of the
* running task. Each worker thread and main thread must get
* the lock first to read or write the attributes of this class.
*/
class TasksState {
public:
   TasksState();
   ~TasksState();

   std::unique_ptr<std::mutex> mutex_;
   std::unique_ptr<std::condition_variable> finished_;
   std::unique_ptr<std::mutex> finishedMutex_;
   IRunnable* runnable_;
   int finished_tasks_;
   int left_tasks_;
   int num_total_tasks_;
};
```

- How will you assign tasks to your worker threads? Should you consider static or dynamic assignment of tasks to threads?
  如何将任务分配给工作线程？应该考虑静态还是动态地将任务分配给线程？

> 并行需要考虑动态地将任务分配给线程，每个线程从共享计数器中获取任务。
>
> 具体代码：

```
void TaskSystemParallelSpawn::threadRun(IRunnable* runnable, int
num_total_tasks, std::mutex* mutex, int* curr_task) {
 int turn = -1;
 while (turn < num_total_tasks) {
     mutex->lock();
     turn = *curr_task;
     *curr_task += 1;
     mutex->unlock();
     if (turn >= num_total_tasks) {
         break;
     }
     runnable->runTask(turn, num_total_tasks);
 }
```

```
}

void TaskSystemParallelSpawn::run(IRunnable* runnable, int
num_total_tasks) {
 std::mutex mutex;
 int curr_task = 0;
 for (int i = 0; i < num_threads_; i++) {
     threads_pool_[i] =
std::thread(&TaskSystemParallelSpawn::threadRun, this, runnable,
num_total_tasks, &mutex, &curr_task);
 }
 for (int i = 0; i < num_threads_; i++) {
     threads_pool_[i].join();
 }
}
```

`threadRun` **函数:**

- `threadRun` 函数是每个工作线程实际执行的任务。它接收一个指向 `IRunnable` 接口实例的指针，一个表示总任务数的整数，一个 `std::mutex` 互斥量的指针以及一个整数指针 `curr_task`，用于跟踪当前任务的索引。
- 在每次迭代中，线程会获取互斥量的锁，然后获取 `curr_task` 的值作为当前任务索引。
- 接着，线程会将 `curr_task` 的值增加 1，然后释放互斥量的锁。
- 然后，线程检查当前任务索引是否超过了总任务数，如果超过则跳出循环；否则，线程调用 `runTask` 方法执行当前任务。
- 这个过程会持续直到所有任务都被执行完成。

`run` **函数:**

- `run` 函数负责启动线程池中的工作线程，并分配任务给它们执行。
- 首先，函数创建一个互斥量和一个整数 `curr_task`，它们作为共享状态传递给工作线程。
- 然后，函数使用 `std::thread` 创建线程池中的每个工作线程，并将 `threadRun` 函数作为线程的入口点。
- 每个工作线程都会接收相同的 `IRunnable` 实例指针和任务数量，以及共享的互斥量和任务索引。
- 最后，函数等待所有工作线程执行完成，然后返回。

- Are there shared variables (internal state of your task execution system) that you need to protect from simultaneous access from multiple threads? You may wish to review our C++ synchronization tutorial for more information on the synchronization primitives in the C++ standard library.
  是否有共享变量（任务执行系统的内部状态）需要防止多个线程同时访问？您可能希望查看我们的C++同步教程，以获得有关C++标准库中同步原语的更多信息。

> 有部分共享变量（`curr_task`、`TasksState` 中的 `mutex_`、`finished_`、`finishedMutex_`、`runnable_`、`finished_tasks_`、`left_tasks_`、`num_total_tasks_`）需要防止多个线程同时访问，主要通过使用多个`std::mutex` 和 `std::condition_variable` 来同步对共享变量的访问，防止多个线程同时访问和修改这些共享变量。

Step 2: Avoid Frequent Thread Creation Using a Thread Pool 第2步：避免使用线程池频繁创建线程

In this step please implement the class `TaskSystemParallelThreadPoolSpinning`.
在这一步中，请实现类 `TaskSystemParallelThreadPoolSpinning` 。

Your implementation in step 1 will incur overhead due to creating threads in each call to `run()`. This overhead is particularly noticeable when tasks are cheap to compute. At this point, we recommend you move to a "thread pool" implementation where your task execution system creates all worker threads up front (e.g., during `TaskSystem` construction, or upon the first call to `run()`).
您在步骤1中的实现将由于在每次调用 `run()` 时创建线程而产生开销。当任务的计算成本很低时，这种开销尤其明显。在这一点上，我们建议您转向“线程池”实现，其中您的任务执行系统预先创建所有工作线程（例如，在 `TaskSystem` 施工期间，或在第一次呼叫 `run()` 时）。

- As a starting implementation we recommend that you design your worker threads to continuously loop, always checking if there is more work to them to perform. (A thread entering a while loop until a condition is true is typically referred to as "spinning".) How might a worker thread determine there is work to do?
  作为一个开始的实现，我们建议您将工作线程设计为连续循环，始终检查是否有更多的工作要执行。（进入while循环直到条件为真的线程通常被称为“自旋”。工作线程如何确定有工作要做?

> 最终代码:
>
> `tasksys.cpp`:

```
TasksState::TasksState()
    : mutex_(std::make_unique<std::mutex>()),
      finished_(std::make_unique<std::condition_variable>()),
      finishedMutex_(std::make_unique<std::mutex>()),
      runnable_(nullptr),
      finished_tasks_(-1),
      left_tasks_(-1),
      num_total_tasks_(-1) {}

TasksState::~TasksState() = default;

const char* TaskSystemParallelThreadPoolSpinning::name() {
return "Parallel + Thread Pool + Spin";
}

TaskSystemParallelThreadPoolSpinning::TaskSystemParallelThreadPoolSpinn
ing(int num_threads)
    : ITaskSystem(num_threads), num_threads_(num_threads), killed(false)
{
state_ = std::make_unique<TasksState>();
threads_pool_ = std::make_unique<std::thread[]>(num_threads);
for (int i = 0; i < num_threads; i++) {
   threads_pool_[i] =
std::thread(&TaskSystemParallelThreadPoolSpinning::spinningThread,
this);
}
}
```

```
TaskSystemParallelThreadPoolSpinning::~TaskSystemParallelThreadPoolSpin
ning() {
killed = true;
for (int i = 0; i < num_threads_; i++) {
   threads_pool_[i].join();
}
}

void TaskSystemParallelThreadPoolSpinning::spinningThread() {
int id;
int total;
while (true) {
   if (killed) break;
   state_->mutex_->lock();
   total = state_->num_total_tasks_;
   id = total - state_->left_tasks_;
   if (id < total) state_->left_tasks_--;
   state_->mutex_->unlock();
   if (id < total) {
       state_->runnable_->runTask(id, total);
       state_->mutex_->lock();
       state_->finished_tasks_++;
       if (state_->finished_tasks_ == total) {
           state_->mutex_->unlock();
           // this lock is necessary to ensure the main thread has gone
to sleep
           state_->finishedMutex_->lock();
           state_->finishedMutex_->unlock();
           state_->finished_->notify_all();
       } else {
           state_->mutex_->unlock();
       }
   }
}
}
```

`tasksys.h`:

```
/*
* TaskSystemParallelThreadPoolSpinning: This class is the student's
* implementation of a parallel task execution engine that uses a
* thread pool. See definition of ITaskSystem in itasksys.h for
* documentation of the ITaskSystem interface.
*/
class TaskSystemParallelThreadPoolSpinning : public ITaskSystem {
public:
   TaskSystemParallelThreadPoolSpinning(int num_threads);
   ~TaskSystemParallelThreadPoolSpinning();
   const char* name() override;
   void run(IRunnable* runnable, int num_total_tasks) override;
   TaskID runAsyncWithDeps(IRunnable* runnable, int num_total_tasks,
                           const std::vector<TaskID>& deps) override;
   void sync() override;

private:
```

```
    void spinningThread();
    std::unique_ptr<TasksState> state_;
    std::unique_ptr<std::thread[]> threads_pool_;
    bool killed;
    int num_threads_;
};
```

- It is now non-trivial to ensure that `run()` implements the required synchronous behavior. How do you need to change the implementation of `run()` to determine that all tasks in the bulk task launch have completed?

  现在，确保 `run()` 实现所需的同步行为是很重要的。您需要如何更改 `run()` 的实现以确定批量任务启动中的所有任务都已完成?

```
void TaskSystemParallelThreadPoolSpinning::run(IRunnable* runnable, int
num_total_tasks) {
 std::unique_lock<std::mutex> lk(*(state_->finishedMutex_));
 state_->mutex_->lock();
 state_->runnable_ = runnable;
 state_->num_total_tasks_ = num_total_tasks;
 state_->left_tasks_ = num_total_tasks;
 state_->finished_tasks_ = 0;
 state_->mutex_->unlock();
 while (state_->finished_tasks_ < num_total_tasks) {
     state_->finished_->wait(lk);
 }
}
```

这段代码确保了在所有任务完成之前，工作线程会一直等待。通过使用条件变量 `state_->finished_` 和互斥量 `state_->finishedMutex_`，线程能够以最有效的方式等待任务完成的通知，并且不会浪费 CPU 时间在空闲的轮询上。

Step 3: Put Threads to Sleep When There is Nothing to Do 第三步：无事可做时让线程休眠

In this step please implement the class `TaskSystemParallelThreadPoolSleeping`.
在这一步中，请实现类 `TaskSystemParallelThreadPoolSleeping` 。

One of the drawbacks of the step 2 implementation is that threads utilize a CPU core's execution resources as they "spin" waiting for something to do. For example, worker threads might loop waiting for new tasks to arrive. As another example, the main thread might loop waiting for the worker threads to complete all tasks so it can return from a call to `run()`. This can hurt performance since CPU resources are used to run these threads even though the threads are not doing useful work.
第2步实现的缺点之一是，线程在"旋转"等待某件事时会利用CPU核心的执行资源。例如，工作线程可能会循环等待新任务到达。作为另一个例子，主线程可能会循环等待工作线程完成所有任务，这样它就可以从对 `run()` 的调用中返回。这可能会影响性能，因为即使线程没有做有用的工作，CPU资源也会用于运行这些线程。

In this part of the assignment, we want you to improve the efficiency of your task system by putting threads to sleep until the condition they are waiting for is met.
在任务的这一部分，我们希望您通过将线程置于睡眠状态直到它们等待的条件得到满足来提高任务系统的效率。

- Your implementation may choose to use condition variables to implement this behavior. Condition variables are a synchronization primitive that enables threads to sleep (and occupy no CPU processing resources) while they are waiting for a condition to exist. Other threads "signal" waiting threads to wake up to see if the condition they were waiting for has been met. For example, your worker threads could be put to sleep if there is no work to be done (so they don't take CPU resources away from threads trying to do useful work). As another example, your main application thread that calls `run()` might want to sleep while it waits for all the tasks in a bulk task launch to be completed by the worker threads. (Otherwise a spinning main thread would take CPU resources away from the worker threads!) Please see our C++ synchronization tutorial for more information on condition variables in C++.
  您的实现可以选择使用条件变量来实现此行为。条件变量是一种同步原语，它使线程在等待条件存在时能够休眠（并且不占用CPU处理资源）。其他线程“发信号”等待线程唤醒，以查看它们等待的条件是否已经满足。例如，如果没有工作要做，可以将工作线程置于睡眠状态（这样它们就不会从尝试做有用工作的线程那里占用CPU资源）。再举一个例子，调用 `run()` 的主应用程序线程可能希望在等待工作线程完成批量任务启动中的所有任务时休眠。（否则，一个旋转的主线程会从工作线程那里拿走CPU资源！）有关C++中条件变量的更多信息，请参阅我们的C++同步教程。
- Your implementation in this part of the assignment may have tricky race conditions to think about. You'll need to consider many possible interleavings of thread behavior.
  您在这部分作业中的实作可能需要考虑棘手的竞态条件。您需要考虑线程行为的许多可能的交错。
- You might want to consider writing additional test cases to exercise your system. **The assignment starter code includes the workloads that the grading script will use to grade the performance of your code, but we will also test the correctness of your implementation using a wider set of workloads that we are not providing in the starter code!**
  您可能需要考虑编写额外的测试用例来测试您的系统。作业启动代码包括评分脚本将用于对代码性能进行评分的工作负载，但我们还将使用启动代码中未提供的更广泛的工作负载集来测试您的实现的正确性！

> `tasksys.cpp`:
>
> ```
> TaskSystemParallelThreadPoolSleeping::TaskSystemParallelThreadPoolSleep
> ing(int num_threads)
>     : ITaskSystem(num_threads), num_threads_(num_threads), killed(false)
> {
> state_ = std::make_unique<TasksState>();
> threads_pool_ = std::make_unique<std::thread[]>(num_threads);
> hasTasks = std::make_unique<std::condition_variable>();
> hasTasksMutex = std::make_unique<std::mutex>();
> for (int i = 0; i < num_threads; i++) {
>    threads_pool_[i] =
> std::thread(&TaskSystemParallelThreadPoolSleeping::sleepingThread,
> this);
> }
> }
> ```

```
TaskSystemParallelThreadPoolSleeping::~TaskSystemParallelThreadPoolSlee
ping() {
killed = true;
hasTasks->notify_all();
for (int i = 0; i < num_threads_; i++) {
   threads_pool_[i].join();
}
}

void TaskSystemParallelThreadPoolSleeping::sleepingThread() {
int id;
int total;
while (true) {
   std::unique_lock<std::mutex> lk(*(hasTasksMutex));
   if (killed) break;
   while (!killed && state_->left_tasks_ <= 0) {
       hasTasks->wait(lk);
   }
   if (killed) break;
   state_->mutex_->lock();
   total = state_->num_total_tasks_;
   id = total - state_->left_tasks_;
   if (id < total) state_->left_tasks_--;
   state_->mutex_->unlock();
   if (id < total) {
       state_->runnable_->runTask(id, total);
       state_->mutex_->lock();
       state_->finished_tasks_++;
       if (state_->finished_tasks_ == total) {
           state_->mutex_->unlock();
           // this lock is necessary to ensure the main thread has gone
to sleep
           state_->finishedMutex_->lock();
           state_->finishedMutex_->unlock();
           state_->finished_->notify_all();
       } else {
           state_->mutex_->unlock();
       }
   }
}
}

void TaskSystemParallelThreadPoolSleeping::run(IRunnable* runnable, int
num_total_tasks) {
std::unique_lock<std::mutex> lk(*(state_->finishedMutex_));
state_->mutex_->lock();
state_->runnable_ = runnable;
state_->num_total_tasks_ = num_total_tasks;
state_->left_tasks_ = num_total_tasks;
state_->finished_tasks_ = 0;
state_->mutex_->unlock();
hasTasks->notify_all();
while (state_->finished_tasks_ < num_total_tasks) {
   state_->finished_->wait(lk);
}
```

```
}
```

`tasksys.h`:

```
/*
* TaskSystemParallelThreadPoolSleeping: This class is the student's
* optimized implementation of a parallel task execution engine that
uses
* a thread pool. See definition of ITaskSystem in
* itasksys.h for documentation of the ITaskSystem interface.
*/
class TaskSystemParallelThreadPoolSleeping : public ITaskSystem {
public:
    TaskSystemParallelThreadPoolSleeping(int num_threads);
    ~TaskSystemParallelThreadPoolSleeping();
    const char* name() override;
    void run(IRunnable* runnable, int num_total_tasks) override;
    TaskID runAsyncWithDeps(IRunnable* runnable, int num_total_tasks,
                            const std::vector<TaskID>& deps) override;
    void sync() override;

private:
    void sleepingThread();
    std::unique_ptr<TasksState> state_;
    std::unique_ptr<std::thread[]> threads_pool_;
    std::unique_ptr<std::condition_variable> hasTasks;
    std::unique_ptr<std::mutex> hasTasksMutex;
    bool killed;
    int num_threads_;
};
```

Part B: Supporting Execution of Task Graphs 支持任务图的执行

In part B of the assignment you will extend your part A task system implementation to support the asynchronous launch of tasks that may have dependencies on previous tasks. These inter-task dependencies create scheduling constraints that your task execution library must respect.
在作业的B部分中，您将扩展A部分任务系统实现，以支持可能依赖于先前任务的任务的异步启动。这些任务间的依赖关系创建了任务执行库必须遵守的调度约束。

The `ITaskSystem` interface has an additional method:
`ITaskSystem` 接口有一个额外的方法：

```
virtual TaskID runAsyncWithDeps(IRunnable* runnable, int num_total_tasks,
                                const std::vector<TaskID>& deps) = 0;
```

`runAsyncWithDeps()` is similar to `run()` in that it also is used to perform a bulk launch of `num_total_tasks` tasks. However, it differs from `run()` in a number of ways...
`runAsyncWithDeps()` 类似于 `run()` ，因为它也用于执行 `num_total_tasks` 任务的批量启动。然而，它在许多方面与 `run()` 不同。

Asynchronous Task Launch 异步任务启动

First, tasks created using `runAsyncWithDeps()` are executed by the task system *asynchronously* with the calling thread. This means that `runAsyncWithDeps()`, should return to the caller *immediately*, even if the tasks have not completed execution. The method returns a unique identifier associated with this bulk task launch.
首先，使用 `runAsyncWithDeps()` 创建的任务由任务系统与调用线程异步执行。这意味着 `runAsyncWithDeps()` 应该立即返回到调用者，即使任务尚未完成执行。该方法返回与此批量任务启动关联的唯一标识符。

The calling thread can determine when the bulk task launch has actually completed by calling `sync()`.
调用线程可以通过调用 `sync()` 来确定批量任务启动实际完成的时间。

```
virtual void sync() = 0;
```

`sync()` returns to the caller **only when the tasks associated with all prior bulk task launches have completed.** For example, consider the following code:
`sync()` 仅当与所有先前批量任务启动相关联的任务已完成时才返回给调用方。例如，请考虑以下代码：

```
// assume taskA and taskB are valid instances of IRunnable...

std::vector<TaskID> noDeps;  // empty vector

ITaskSystem *t = new TaskSystem(num_threads);

// bulk launch of 4 tasks
TaskID launchA = t->runAsyncWithDeps(taskA, 4, noDeps);

// bulk launch of 8 tasks
TaskID launchB = t->runAsyncWithDeps(taskB, 8, noDeps);

// at this point tasks associated with launchA and launchB
// may still be running

t->sync();

// at this point all 12 tasks associated with launchA and launchB
// are guaranteed to have terminated
```

As described in the comments above, the calling thread is not guaranteed tasks from previous calls to `runAsyncWithDeps()` have completed until the thread calls `sync()`. To be precise, `runAsyncWithDeps()` tells your task system to perform a new bulk task launch, but your implementation has the flexibility to execute these tasks at any time prior to the next call to `sync()`. Note that this specification means there is no guarantee that your implementation performs tasks from launchA prior to starting tasks from launchB!
如上面的注释中所述，调用线程不保证从先前对 `runAsyncWithDeps()` 的调用开始的任务已经完成，直到线程调用 `sync()` 。确切地说， `runAsyncWithDeps()` 告诉你的任务系统执行一个新的批量任务启动，但是你的实现可以在下一次调用 `sync()` 之前的任何时候灵活地执行这些任务。请注意，此规范意味着不能保证您的实现在从launchB启动任务之前从launchA执行任务！

Support for Explicit Dependencies 支持显式重复

The second key detail of `runAsyncWithDeps()` is its third argument: a vector of TaskID identifiers that must refer to previous bulk task launches using `runAsyncWithDeps()`. This vector specifies what prior tasks the tasks in the current bulk task launch depend on. **Therefore, your task runtime cannot begin execution of any task in the current bulk task launch until all tasks from the launches given in the dependency vector are complete!** For example, consider the following example:
`runAsyncWithDeps()` 的第二个关键细节是它的第三个参数：一个TaskID标识符向量，必须引用以前使用 `runAsyncWithDeps()` 启动的批量任务。此向量指定当前批量任务启动中的任务所依赖的先前任务。因此，在依赖关系向量中给定的启动中的所有任务完成之前，任务运行时不能开始执行当前批量任务启动中的任何任务！例如，考虑以下示例：

```
std::vector<TaskID> noDeps;  // empty vector
std::vector<TaskID> depOnA;
std::vector<TaskID> depOnBC;

ITaskSystem *t = new TaskSystem(num_threads);

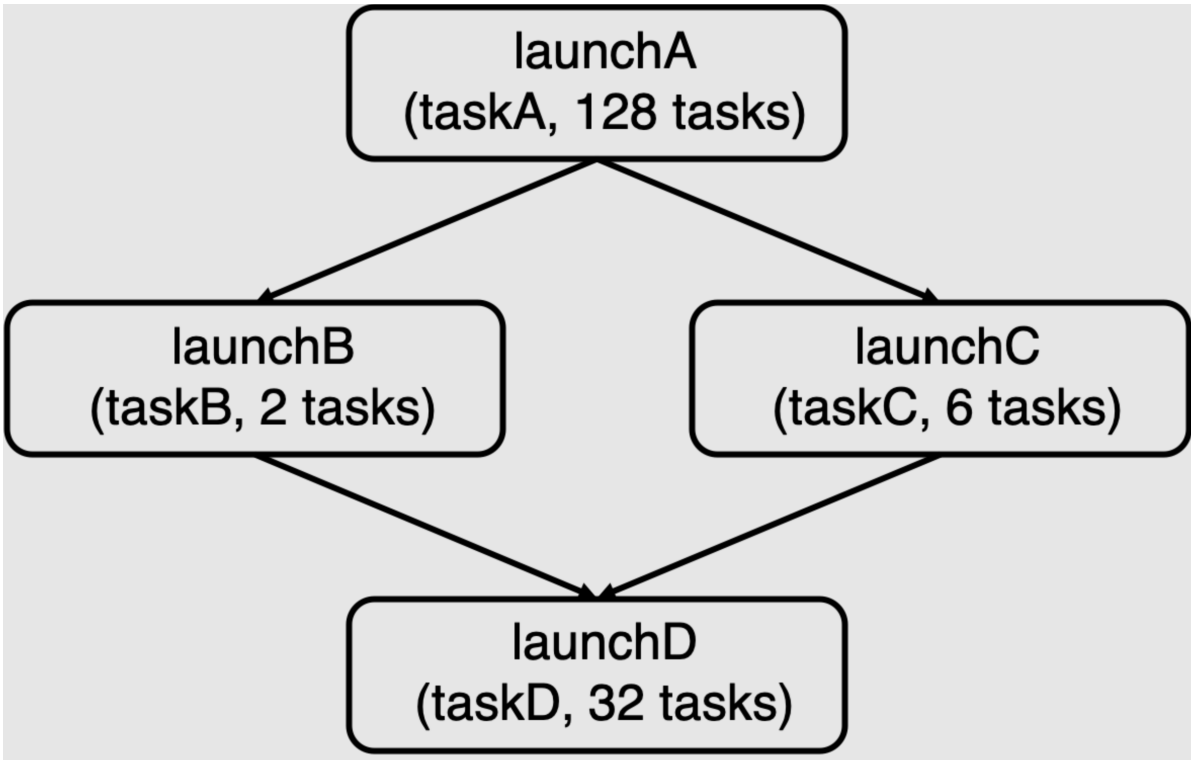
TaskID launchA = t->runAsyncWithDeps(taskA, 128, noDeps);
depOnA.push_back(launchA);

TaskID launchB = t->runAsyncWithDeps(taskB, 2, depOnA);
TaskID launchC = t->runAsyncWithDeps(taskC, 6, depOnA);
depOnBC.push_back(launchB);
depOnBC.push_back(launchC);

TaskID launchD = t->runAsyncWithDeps(taskD, 32, depOnBC);
t->sync();
```

The code above features four bulk task launches (taskA: 128 tasks, taskB: 2 tasks, taskC: 6 tasks, taskD: 32 tasks). Notice that the launch of taskB and of taskC depend on taskA. The bulk launch of taskD (`launchD`) depends on the results of both `launchB` and `launchC`. Therefore, while your task runtime is allowed to process tasks associated with `launchB` and `launchC` in any order (including in parallel), all tasks from these launches must begin executing after the completion of tasks from `launchA`, and they must complete before your runtime can begin executing any task from `launchD`.
上面的代码有四个批量任务启动（taskA：128个任务，taskB：2个任务，taskC：6个任务，taskD：32个任务）。注意，taskB和taskC的启动依赖于taskA。taskD（`launchD`）的批量启动取决于 `launchB` 和 `launchC` 的结果。因此，虽然允许任务运行时以任何顺序（包括并行）处理与 `launchB` 和 `launchC` 关联的任务，但这些启动中的所有任务必须在 `launchA` 中的任务完成后开始执行，并且必须在运行时开始执行 `launchD` 中的任何任务之前完成。

We can illustrate these dependencies visually as a **task graph**. A task graph is a directed acyclic graph (DAG), where nodes in the graph correspond to bulk task launches, and an edge from node X to node Y indicates a dependency of Y on the output of X. The task graph for the code above is:
我们可以将这些依赖性直观地表示为任务图。任务图是有向无环图（DAG），其中图中的节点对应于批量任务启动，并且从节点X到节点Y的边指示Y对X的输出的依赖性。上面代码的任务图是：



Notice that if you were running the example above on a Myth machine with eight execution contexts, the ability to schedule the tasks from `launchB` and `launchC` in parallel might be quite useful, since neither bulk task launch on its own is sufficient to use all the execution resources of the machine.
请注意，如果您在具有八个执行上下文的Myth机器上运行上面的示例，那么并行调度来自 `launchB` 和 `launchC` 的任务的能力可能非常有用，因为单独的批量任务启动都不足以使用机器的所有执行资源。

Testing 测试

All of the tests with postfix `Async` should be used to test part B. The subset of tests included in the grading harness are described in `tests/README.md`, and all tests can be found in `tests/tests.h` and are listed in `tests/main.cpp`. To debug correctness, we've provided a small test `simple_test_async`. Take a look at the `simpleTest` function in `tests/tests.h`. `simple_test_async` should be small enough to debug using print statements or breakpoints inside `simpleTest`.
所有带有后缀 `Async` 的测试均应用于测试部分B。分级线束中包含的测试子集在 `tests/README.md` 中描述，所有测试可在 `tests/tests.h` 中找到，并在 `tests/main.cpp` 中列出。为了调试正确性，我们提供了一个小测试 `simple_test_async` 。看看 `tests/tests.h` 中的 `simpleTest` 函数。`simple_test_async` 应该足够小，以便在 `simpleTest` 中使用print语句或断点进行调试。

We encourage you to create your own tests. Take a look at the existing tests in `tests/tests.h` for inspiration. We have also included a skeleton test composed of `class YourTask` and function `yourTest()` for you to build on if you so choose. For the tests you do create, make sure to add them to the list of tests and test names in `tests/main.cpp`, and adjust the variable `n_tests` accordingly. Please note that while you will be able to run your own tests with your solution, you will not be able to compile the reference solution to run your tests.
我们鼓励您创建自己的测试。看看 `tests/tests.h` 中的现有测试以获得灵感。我们还包含了一个由 `class YourTask` 和函数 `yourTest()` 组成的框架测试，如果您选择的话，可以在此基础上进行构建。对于您创建的测试，请确保将它们添加到 `tests/main.cpp` 中的测试和测试名称列表中，并相应地调整变量 `n_tests` 。请注意，虽然您可以使用解决方案运行自己的测试，但您将无法编译参考解决方案来运行测试。

What You Need to Do 你需要做的

You must extend your task system implementation that uses a thread pool (and sleeps) from part A to correctly implement `TaskSystemParallelThreadPoolSleeping::runAsyncWithDeps()` and `TaskSystemParallelThreadPoolSleeping::sync()`. We also expect you to create at least one test, which can test either correctness or performance. See the `Testing` section above for more information. As a clarification, you will *need* to describe your own tests in the writeup, but our autograder will *NOT* test your test. **You do not need to implement the other `TaskSystem` classes in Part B.**
您必须扩展您的任务系统实现，它使用A部分中的线程池（和休眠）来正确实现 `TaskSystemParallelThreadPoolSleeping::runAsyncWithDeps()` 和 `TaskSystemParallelThreadPoolSleeping::sync()` 。我们还希望您至少创建一个测试，可以测试正确性或性能。更多信息请参见上文的 `Testing` 部分。作为澄清，你需要在文章中描述你自己的测试，但是我们的自动评分器不会测试你的测试。您不需要实现B部分中的其他 `TaskSystem` 类。

As with Part A, we offer you the following tips to get started:
与A部分一样，我们为您提供以下入门提示：

- It may be helpful to think about the behavior of `runAsyncWithDeps()` as pushing a record corresponding to the bulk task launch, or perhaps records corresponding to each of the tasks in the bulk task launch onto a "work queue". Once the record to work to do is in the queue, `runAsyncWithDeps()` can return to the caller.
  将 `runAsyncWithDeps()` 的行为视为将对应于批量任务启动的记录或者可能对应于批量任务启动中的每个任务的记录推送到“工作队列”上可能是有帮助的。一旦要工作的记录在队列中，`runAsyncWithDeps()` 就可以返回给调用者。
- The trick in this part of the assignment is performing the appropriate bookkeeping to track dependencies. What must be done when all the tasks in a bulk task launch complete? (This is the point when new tasks may become available to run.)
  这部分任务的技巧是执行适当的簿记来跟踪依赖关系。当批量任务启动中的所有任务完成时，必须执行什么操作？(This是新任务可以运行的时间点。)
- It can be helpful to have two data structures in your implementation: (1) a structure representing tasks that have been added to the system via a call to `runAsyncWithDeps()`, but are not yet ready to execute because they depend on tasks that are still running (these tasks are "waiting" for others to finish) and (2) a "ready queue" of tasks that are not waiting on any prior tasks to finish and can safely be run as soon as a worker thread is available to process them.
  在你的实现中有两个数据结构会很有帮助：（1）表示已经通过对 `runAsyncWithDeps()` 的调用添加到系统的任务的结构，但尚未准备好执行，因为它们依赖于仍在运行的任务（这些任务正在“等待”其他任务完成）和（2）“就绪队列”不等待任何先前任务完成的任务，并且可以在工作线程可用于处理它们时安全地运行。
- You need not worry about integer wrap around when generating unique task launch ids. We will not hit your task system with over 2^31 bulk task launches.
  在生成唯一的任务启动id时，您不必担心整数回绕。我们不会用超过2^31个批量任务启动来攻击你的任务系统。
- You can assume all programs will either call only `run()` or only `runAsyncWithDeps()`; that is, you do not need to handle the case where a `run()` call needs to wait for all proceeding calls to `runAsyncWithDeps()` to finish. Note that this assumption means you can implement `run()` using appropriate calls to `runAsyncWithDeps()` and `sync()`.
  你可以假设所有的程序要么只调用 `run()` ，要么只调用 `runAsyncWithDeps()` ;也就是说，你不需要处理这样的情况：一个 `run()` 调用需要等待所有对 `runAsyncWithDeps()` 的调用完成。请

注意，这个假设意味着您可以使用对 `runAsyncWithDeps()` 和 `sync()` 的适当调用来实现 `run()` 。

- You can assume the only multithreading going on is the multiple threads created by/used by your implementation. That is, we won't be spawning additional threads and calling your implementation from those threads.
  您可以假设唯一正在进行的多线程是由您的实现创建/使用的多个线程。也就是说，我们不会产生额外的线程并从这些线程调用您的实现。

Implement your part B implementation in the `part_b/` sub-directory to compare to the correct reference implementation (`part_b/runtasks_ref_\*`).
在 `part_b/` 子目录中实现部分B实现，以与正确的参考实现（ `part_b/runtasks_ref_\*` ）进行比较。

> `tasksys.cpp`:
>
> ```
> TaskSystemParallelThreadPoolSleeping::TaskSystemParallelThreadPoolSleeping(i
> nt num_threads): ITaskSystem(num_threads) {
> //
> // TODO: CS149 student implementations may decide to perform setup
> // operations (such as thread pool construction) here.
> // Implementations are free to add new class member variables
> // (requiring changes to tasksys.h).
> // 初始化所有成员变量和线程池
> _threads_pool = std::make_unique<std::thread[]>(num_threads);
> for (int i = 0; i < _num_Threads; i++) {
>    _threads_pool[i] =
> std::thread(&TaskSystemParallelThreadPoolSleeping::workerThread, this);
> }
> }
>
> TaskSystemParallelThreadPoolSleeping::~TaskSystemParallelThreadPoolSleeping(
> ) {
> //
> // TODO: CS149 student implementations may decide to perform cleanup
> // operations (such as thread pool shutdown construction) here.
> // Implementations are free to add new class member variables
> // (requiring changes to tasksys.h).
> // 释放所有资源
> _killed = true;
> hasTasks.notify_all(); // Notify all threads to wake up and finish.
> for (int i = 0; i < _num_Threads; i++) {
>    if (_threads_pool[i].joinable()) {
>        _threads_pool[i].join();
>    }
> }
> }
>
> void TaskSystemParallelThreadPoolSleeping::run(IRunnable* runnable, int
> num_total_tasks) {
> std::vector<TaskID> noDeps;
> runAsyncWithDeps(runnable, num_total_tasks, noDeps);
> sync();
> }
> ```

```
void TaskSystemParallelThreadPoolSleeping::workerThread() {
while (!_killed) {
   WaitingTask task(0, -1, nullptr, 0);
   {
       std::unique_lock<std::mutex> lock(_waiting_queue_mutex);
       hasTasks.wait(lock, [this] { return _killed ||
!_waiting_queue.empty(); });
       if (_killed) break;
       if (!_waiting_queue.empty()) {
           task = std::move(_waiting_queue.front());
           _waiting_queue.pop();
       }
   }
   if (task.runnable) {
       for (int i = 0; i < task.num_total_tasks; i++) {
           task.runnable->runTask(i, task.num_total_tasks);
       }
       {
           std::unique_lock<std::mutex> lock(_meta_data_mutex);
           _finished_TaskID = task.id;
           _finished.notify_all();
       }
   }
}
}

TaskID TaskSystemParallelThreadPoolSleeping::runAsyncWithDeps(IRunnable*
runnable, int num_total_tasks,
                                                              const
std::vector<TaskID>& deps) {
TaskID dependency = deps.empty() ? -1 : *std::max(deps.begin(), deps.end());
WaitingTask task(_next_TaskID, dependency, runnable, num_total_tasks);
{
   std::unique_lock<std::mutex> lock(_waiting_queue_mutex);
   _waiting_queue.push(std::move(task));
}
hasTasks.notify_all();
return _next_TaskID++;
}


void TaskSystemParallelThreadPoolSleeping::sync() {
std::unique_lock<std::mutex> lock(_meta_data_mutex);
_finished.wait(lock, [this] { return _waiting_queue.empty() && (_next_TaskID
== _finished_TaskID); });
}
```

`tasksys.h`:

```
#ifndef _TASKSYS_H
#define _TASKSYS_H

#include "itasksys.h"
#include <thread>
#include <mutex>
```

```
#include <atomic>
#include <condition_variable>
#include <queue>
#include <iostream>
#include <memory>
#include <vector>

/*
* TaskSystemSerial: This class is the student's implementation of a
* serial task execution engine. See definition of ITaskSystem in
* itasksys.h for documentation of the ITaskSystem interface.
*/
class TaskSystemSerial : public ITaskSystem {
public:
    TaskSystemSerial(int num_threads);
    ~TaskSystemSerial();
    const char* name() override;
    void run(IRunnable* runnable, int num_total_tasks) override;
    TaskID runAsyncWithDeps(IRunnable* runnable, int num_total_tasks, const
std::vector<TaskID>& deps) override;
    void sync() override;
};

/*
* TaskSystemParallelSpawn: This class is the student's implementation of a
* parallel task execution engine that spawns threads in every run() call.
* See definition of ITaskSystem in itasksys.h for documentation of the
ITaskSystem interface.
*/
class TaskSystemParallelSpawn : public ITaskSystem {
public:
    TaskSystemParallelSpawn(int num_threads);
    ~TaskSystemParallelSpawn();
    const char* name() override;
    void run(IRunnable* runnable, int num_total_tasks) override;
    TaskID runAsyncWithDeps(IRunnable* runnable, int num_total_tasks, const
std::vector<TaskID>& deps) override;
    void sync() override;

private:
    void threadRun(IRunnable* runnable, int num_total_tasks, std::mutex*
mutex, int* curr_task);
    std::unique_ptr<std::thread[]> threads_pool_;
    int num_threads_;
};

/*
* WaitingTask: This class represents a task with a dependency.
*/
class WaitingTask {
public:
    TaskID id;
    TaskID dependency;
    IRunnable* runnable;
    int num_total_tasks;
```

```
    WaitingTask(TaskID id, TaskID dependency, IRunnable* runnable, int
num_total_tasks)
            : id(id), dependency(dependency), runnable(runnable),
num_total_tasks(num_total_tasks) {}
};

/*
* TasksState: This class is used to record the state of the running task.
*/
class TasksState {
public:
    TasksState();
    ~TasksState();

    std::unique_ptr<std::mutex> mutex_;
    std::unique_ptr<std::condition_variable> finished_;
    std::unique_ptr<std::mutex> finishedMutex_;
    IRunnable* runnable_;
    int finished_tasks_;
    int left_tasks_;
    int num_total_tasks_;
};

/*
* TaskSystemParallelThreadPoolSpinning: This class is the student's
implementation of a
* parallel task execution engine that uses a thread pool.
* See definition of ITaskSystem in itasksys.h for documentation of the
ITaskSystem interface.
*/
class TaskSystemParallelThreadPoolSpinning : public ITaskSystem {
public:
    TaskSystemParallelThreadPoolSpinning(int num_threads);
    ~TaskSystemParallelThreadPoolSpinning();
    const char* name() override;
    void run(IRunnable* runnable, int num_total_tasks) override;
    TaskID runAsyncWithDeps(IRunnable* runnable, int num_total_tasks, const
std::vector<TaskID>& deps) override;
    void sync() override;

private:
    void spinningThread();
    std::unique_ptr<TasksState> state_;
    std::unique_ptr<std::thread[]> threads_pool_;
    bool killed;
    int num_threads_;
};

/*
* TaskSystemParallelThreadPoolSleeping: This class is the student's
optimized implementation of a
* parallel task execution engine that uses a thread pool.
* See definition of ITaskSystem in itasksys.h for documentation of the
ITaskSystem interface.
```

```
*/
class TaskSystemParallelThreadPoolSleeping : public ITaskSystem {
public:
    TaskSystemParallelThreadPoolSleeping(int num_threads);
    ~TaskSystemParallelThreadPoolSleeping();
    const char* name() override;
    void run(IRunnable* runnable, int num_total_tasks) override;
    TaskID runAsyncWithDeps(IRunnable* runnable, int num_total_tasks, const
std::vector<TaskID>& deps) override;
    void sync() override;

private:
    void workerThread();

    int _num_Threads;
    bool _killed;
    TaskID _finished_TaskID;
    TaskID _next_TaskID;
    std::condition_variable _finished;
    std::mutex _waiting_queue_mutex;
    std::mutex _ready_queue_mutex;
    std::mutex _meta_data_mutex;
    std::unique_ptr<std::thread[]> _threads_pool;
    std::queue<WaitingTask> _waiting_queue;
    std::condition_variable hasTasks;
};

#endif
```

对于测试，使用了如下测试框架和工具：

- **Google Test**：一个流行的 C++ 测试框架，用于编写单元测试。
- **Chrono**：C++ 标准库中的时间工具，用于性能测试中的计时。

具体实现：

1. 正确性测试：

```
#include <gtest/gtest.h>
#include "tasksys.h"

class SimpleRunnable : public IRunnable {
public:
    SimpleRunnable(int* counter, std::mutex* mutex)
        : counter_(counter), mutex_(mutex) {}

    void runTask(int task_id, int num_total_tasks) override {
        std::lock_guard<std::mutex> lock(*mutex_);
        (*counter_)++;
    }

private:
    int* counter_;
    std::mutex* mutex_;
```

```
};

TEST(TaskSystemSerialTest, Correctness) {
    int counter = 0;
    std::mutex mutex;
    SimpleRunnable runnable(&counter, &mutex);
    TaskSystemSerial system(1);
    system.run(&runnable, 10);

    EXPECT_EQ(counter, 10);
}

TEST(TaskSystemParallelSpawnTest, Correctness) {
    int counter = 0;
    std::mutex mutex;
    SimpleRunnable runnable(&counter, &mutex);
    TaskSystemParallelSpawn system(4);
    system.run(&runnable, 10);

    EXPECT_EQ(counter, 10);
}

TEST(TaskSystemParallelThreadPoolSpinningTest, Correctness) {
    int counter = 0;
    std::mutex mutex;
    SimpleRunnable runnable(&counter, &mutex);
    TaskSystemParallelThreadPoolSpinning system(4);
    system.run(&runnable, 10);

    EXPECT_EQ(counter, 10);
}

TEST(TaskSystemParallelThreadPoolSleepingTest, Correctness) {
    int counter = 0;
    std::mutex mutex;
    SimpleRunnable runnable(&counter, &mutex);
    TaskSystemParallelThreadPoolSleeping system(4);
    system.run(&runnable, 10);

    EXPECT_EQ(counter, 10);
}
```

创建一个简单的任务类 `SimpleRunnable`，它会在每个任务运行时递增一个计数器。通过在四种不同的任务系统上运行这些任务，验证计数器是否等于任务总数。

2. 性能测试

```
#include <gtest/gtest.h>
#include <chrono>
#include "tasksys.h"

class HeavyRunnable : public IRunnable {
public:
```

```
    void runTask(int task_id, int num_total_tasks) override {
        for (volatile int i = 0; i < 1000000; i++); // 模拟繁重的计算任务
    }
};

void runPerformanceTest(ITaskSystem& system, int num_total_tasks) {
    HeavyRunnable runnable;
    auto start = std::chrono::high_resolution_clock::now();
    system.run(&runnable, num_total_tasks);
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed = end - start;
    std::cout << "Time taken: " << elapsed.count() << " seconds" <<
std::endl;
}

TEST(TaskSystemPerformanceTest, SerialPerformance) {
    TaskSystemSerial system(1);
    runPerformanceTest(system, 100);
}

TEST(TaskSystemPerformanceTest, ParallelSpawnPerformance) {
    TaskSystemParallelSpawn system(4);
    runPerformanceTest(system, 100);
}

TEST(TaskSystemPerformanceTest, ParallelThreadPoolSpinningPerformance)
{
    TaskSystemParallelThreadPoolSpinning system(4);
    runPerformanceTest(system, 100);
}

TEST(TaskSystemPerformanceTest, ParallelThreadPoolSleepingPerformance)
{
    TaskSystemParallelThreadPoolSleeping system(4);
    runPerformanceTest(system, 100);
}
```

创建一个耗时任务类 `HeavyRunnable`，模拟繁重的计算任务，来测量在不同任务系统上运行这些任务所需的时间，并输出执行时间。

Writeup Handin

1. Describe your task system implementation (1 page is fine). In additional to a general description of how it works, please make sure you address the following questions:
   描述您的任务系统实现（1页即可）。除了对它如何工作的一般描述外，请确保您解决了以下问题：

- How did you decide to manage threads? (e.g., did you implement a thread pool?)
  您是如何决定管理线程的？(e.g.,你实现了线程池吗？)
- How does your system assign tasks to worker threads? Did you use static or dynamic assignment?
  系统如何将任务分配给工作线程？你是用静态赋值还是动态赋值？

- How did you track dependencies in Part B to ensure correct execution of task graphs?
  您如何跟踪部分B中的依赖关系以确保任务图的正确执行?

  > 我实现的任务系统旨在并行执行任务，有效管理资源，并在存在依赖关系时确保任务按正确顺序执行。该系统结合了简单的串行执行和基于线程池的执行来优化性能。任务系统有几个实现版本，每个版本都针对不同的使用场景进行了优化：串行执行、每次运行时生成线程的并行执行以及线程空闲时会自旋或休眠的线程池并行执行。
  >
  > ### 线程管理
  >
  > 在并行实现中，我使用了线程池来管理线程。线程池是预先创建并重复使用的工作线程集合，用于执行多个任务。这种方法避免了反复创建和销毁线程的开销，这对性能来说是非常昂贵的。
  >
  > - **线程池创建**：当实例化 `TaskSystemParallelThreadPoolSleeping`（或 `TaskSystemParallelThreadPoolSpinning`）时，它根据构造时指定的线程数量初始化一组固定的线程。这些线程启动并进入等待状态，准备接收任务。
  > - **资源管理**：通过保留一组线程，系统确保了资源的有效利用，减少了与线程创建和销毁相关的开销。此外，使用条件变量可以高效地管理线程，使它们只在有任务要处理时被唤醒。
  >
  > ### 任务分配
  >
  > 任务动态分配给工作线程，这提供了灵活性和处理不同大小和复杂性工作负载的效率。
  >
  > - **动态分配**：在线程池实现中，任务被推入队列。工作线程从队列中获取任务并执行。这种方法平衡了各线程的负载，确保所有线程都得到有效利用，没有线程成为瓶颈。
  > - **队列管理**：任务队列由互斥锁保护，以确保在任务插入和移除时的线程安全。条件变量用于通知工作线程有新任务可用，从而使它们从等待状态中唤醒。
  >
  > ### 依赖关系跟踪
  >
  > 在B部分中，任务系统需要处理任务之间的依赖关系，以确保任务按正确的顺序执行。
  >
  > - **依赖管理**：每个任务可以指定一个依赖列表，这些依赖任务必须先完成才能开始执行当前任务。系统通过任务ID跟踪这些依赖关系。
  > - **任务图执行**：当提交一个有依赖关系的任务时，它会被放入等待队列。系统检查依赖关系，并且只有当所有依赖关系都满足时，才将任务移至准备队列。这确保了任务按正确顺序执行。
  > - **同步和等待**：系统使用条件变量来管理同步。工作线程等待任务变为可用，任务等待其依赖关系完成。这种机制确保任务按依赖关系执行，防止竞争条件并确保正确执行。

2. In Part A, you may have noticed that simpler task system implementations (e.g., a completely serial implementation, or the spawn threads every launch implementation), perform as well as or sometimes better than the more advanced implementations. Please explain why this is the case, citing certain tests as examples. For example, in what situations did the sequential task system implementation perform best? Why? In what situations did the spawn-every-launch implementation perform as well as the more advanced parallel implementations that use a thread pool? When does it not?
   在A部分中，您可能已经注意到，更简单的任务系统实现（例如，完全串行的实现或每次启动实现的派生线程）与更高级的实现一样好地执行，或者有时比更高级的实现更好地执行。请以某些测试为例，解释为什么会出现这种情况。例如，在什么情况下，顺序任务系统的执行效果最好？为什么要这样？在什么情况下spawn-every-launch实现的性能与使用线程池的更高级的并行实现一样好？什么时候不是？

顺序任务系统（Serial Implementation）

顺序任务系统执行效果最好的情况

1. **任务数量少且开销大**：当任务的数量非常少（例如只有几个）且每个任务的开销较大时，顺序执行往往表现最佳。这是因为多线程实现的开销（如线程创建、调度和上下文切换）相对于任务执行时间来说变得显著。具体来说，测试中如果每个任务都需要大量计算，而总任务数只有几个，那么顺序执行避免了多线程带来的额外开销，表现会更好。

   示例测试：假设有一个计算密集型任务集，每个任务需要几秒钟的计算时间，总任务数少于5个。在这种情况下，顺序任务系统会表现出色。

2. **无并行计算需求**：在某些情况下，任务本身是无法并行化的，或者并行化并不会带来性能提升。比如一些依赖序列化执行的任务，这种情况下顺序任务系统是最合适的选择。

   示例测试：执行一系列必须按顺序处理的数据操作，如从文件中读取数据、处理并写回文件。

原因

顺序任务系统避免了线程创建、销毁、调度和同步的开销。在上述情况下，这些开销会占用大量时间，而直接顺序执行则更高效。

每次启动都生成线程（Spawn-Every-Launch Implementation）

与更高级实现表现一样好的情况

1. **任务数量中等且独立**：当任务数量适中（例如几十个）且每个任务独立且相对均匀时，每次启动都生成线程的实现可能表现得与使用线程池的更高级并行实现一样好。任务数量足够多，能充分利用多核处理器的性能，但又不至于使线程创建的开销变得过高。

   示例测试：假设有50个独立的任务，每个任务需要100毫秒的计算时间。在这种情况下，生成线程的开销（几十个线程）被均摊在任务执行时间内，不会显著影响性能。

性能较差的情况

1. **任务数量庞大**：当任务数量非常庞大（例如数千个）时，每次启动都生成线程的实现会表现不佳，因为线程创建和销毁的开销变得显著。

   示例测试：有2000个任务，每个任务需要5毫秒的计算时间。在这种情况下，生成2000个线程的开销将非常高，导致性能大幅下降。

2. **任务开销极小**：当任务开销非常小（如每个任务只需要微秒级时间）时，每次生成线程的实现同样表现不佳。线程管理的开销在这种情况下远远超过任务执行的开销。

   示例测试：有1000个任务，每个任务只需1毫秒的计算时间。生成1000个线程的开销在这种情况下将显得非常不划算。

原因

每次启动都生成线程的实现中，线程创建和销毁的开销在任务数量适中且任务执行时间足够长时，可以被均摊和隐藏。然而，当任务数量庞大或者任务开销极小时，线程管理的开销会显著影响整体性能。

高级并行实现（Thread Pool Implementation）

表现最佳的情况

1. **大量任务且任务开销小**：当任务数量非常多且每个任务的开销相对较小时，使用线程池的实现会表现最佳。线程池避免了反复创建和销毁线程的开销，线程可以重复使用，大大提高了效率。

   示例测试：有2000个任务，每个任务需要1毫秒的计算时间。在这种情况下，线程池实现能够高效管理线程，并行处理这些任务。

2. **任务有依赖关系**：线程池实现可以有效管理任务之间的依赖关系，确保任务按正确顺序执行，而不会有太多的线程管理开销。

   示例测试：执行有复杂依赖关系的任务图，每个任务依赖其他多个任务的完成。这种情况下，线程池实现能够更好地管理和调度这些任务。

原因

线程池实现通过重用线程来减少线程管理的开销，并提供更高效的任务调度和同步机制。在任务数量大且每个任务执行时间短的情况下，线程池实现能够显著提升性能。此外，线程池实现能够有效处理任务依赖关系，确保任务按正确顺序执行。

总结

不同任务系统实现适用于不同的场景。顺序任务系统在任务数量少且任务开销大时表现最佳；每次启动生成线程的实现适用于任务数量适中且任务独立的情况；而线程池实现则在大量小任务以及存在复杂依赖关系的任务中表现最佳。理解这些实现的适用场景和性能特点，有助于在不同应用中选择合适的任务系统实现。

3. Describe one test that you implemented for this assignment. What does the test do, what is it meant to check, and how did you verify that your solution to the assignment did well on your test? Did the result of the test you added cause you to change your assignment implementation?
描述一个你为这个作业所做的测试。测试做了什么，它的目的是检查什么，你是如何验证你的作业解决方案在测试中表现良好的？您添加的测试的结果是否导致您更改分配实现？

测试目的

这个测试的目的是检查任务系统在处理大量独立小任务时的性能和正确性。独立小任务的场景在实际应用中非常常见，例如处理大量小文件、并行化图像处理中的每个像素块等。因此，我们需要确保任务系统在这种情况下能够高效地管理和调度任务，并正确执行每个任务。

测试设计

我设计了一个简单的任务，每个任务执行非常短的时间（例如，1毫秒）。为了模拟实际情况，我设置了一个任务数量较大的场景（例如，1000个任务）。这个测试的具体步骤如下：

1. **任务定义**：定义一个简单的任务，该任务仅执行一个固定的短时间的计算。
2. **任务数量**：设定任务数量为1000个。
3. **任务系统初始化**：初始化任务系统，包括顺序任务系统和并行任务系统（如每次启动生成线程的实现和线程池实现）。
4. **任务执行**：使用不同的任务系统执行这些任务，记录每种任务系统的执行时间。
5. **结果验证**：验证任务系统执行后的结果是否正确（例如，所有任务都被正确执行）。
6. **性能评估**：比较不同任务系统的执行时间，评估其性能。

测试结果

通过这个测试，我希望得到以下几方面的验证结果：

1. **正确性**：所有任务系统（顺序、每次启动生成线程、线程池实现）在处理1000个任务时都能正确执行每个任务，即每个任务都被调用并完成。
2. **性能对比**：比较不同任务系统的执行时间，评估其性能差异。

是否导致更改分配实现

通过测试结果，我发现线程池实现的性能优势明显，因此可以进一步优化和改进线程池实现，以确保在各种任务场景中都能提供最佳性能。具体改进措施可能包括：

1. **优化任务调度**：通过改进任务队列的管理和调度算法，进一步减少任务等待时间。
2. **减少锁竞争**：在任务队列的访问和任务执行的同步操作中，尽量减少锁的竞争和上下文切换开销。
3. **调整线程池大小**：根据任务的特点和系统资源情况，动态调整线程池的大小，以实现资源利用最大化。